

云原生架构-概念讲解

1. 先给结论

云原生架构不是“把系统部署到云服务器上”，也不是“用了 Docker 就叫云原生”。

一句话理解：

云原生架构 = 让应用天然适合在云环境中运行、扩缩容、发布、治理、观测和恢复的一套架构原则与技术体系。

更考试化的表达：

云原生架构是基于云原生技术的一组架构原则和设计模式的集合，通过服务化、容器化、自动化、可观测、弹性、韧性、零信任等方式，把大量非业务能力交给云平台 and 基础设施承载，使应用具备轻量、敏捷、高度自动化、可伸缩和可持续演进的特点。

最重要的理解：

业务代码只写业务；
弹性、发布、治理、观测、安全、容错等通用能力尽量平台化、自动化、基础设施化。

2. 为什么会出现云原生

传统企业应用通常这样建设：

一套应用
一组服务器
一个数据库
人工部署
人工扩容
人工排障

这种方式在业务规模小、发布频率低时可以工作。但当系统面对互联网流量、快速迭代、多团队协作、峰值活动和复杂运维时，会暴露很多问题：

- 部署慢：上线依赖人工操作，环境差异容易出错。
- 扩容慢：流量高峰来了，需要手工加机器、改配置。
- 故障恢复慢：实例异常、节点宕机后，需要人工发现和处理。
- 系统耦合重：一个模块变更可能影响整个应用。
- 可观测性差：出了问题不知道是哪条链路、哪个服务、哪个数据库调用慢。
- 资源利用率低：为峰值长期准备机器，平时大量资源闲置。

云原生要解决的就是这些问题。

云原生的核心出发点是：

充分利用云计算的弹性、自动化、托管服务和分布式基础设施能力。

因此，云原生不是单一技术，而是一套适合云环境的软件架构方法。

3. 云原生和普通上云的区别

很多题会故意混淆“上云”和“云原生”。

对比项	普通上云	云原生
核心动作	把原系统搬到云服务器	按云环境重新设计架构和交付方式
应用形态	可能仍是单体应用	多采用服务化、微服务、Serverless 等
部署方式	人工部署或脚本部署	容器化、声明式、自动化部署
扩缩容	多依赖人工	可按负载自动伸缩
故障恢复	人工发现、人工处理	健康检查、自动拉起、故障隔离
可观测性	日志为主，链路不清晰	Logging、Tracing、Metrics 组合
发布方式	全量发布，风险较大	蓝绿、灰度、金丝雀发布
基础设施	机器为中心	平台和服务为中心

记忆：

上云是位置变化；
云原生是架构、交付、运维和治理方式变化。

4. 云原生架构的七大原则

软考教材中云原生架构原则非常重要，选择题和案例题都可能考。

4.1 服务化原则

服务化原则是指把不同生命周期、不同变化频率的模块拆分出来，形成相对独立的服务。

它解决的问题：

- 模块边界不清。
- 一个功能修改影响整个系统。
- 不同团队无法独立开发、测试、发布。
- 某个热点模块无法单独扩容。

考试关键词：

模块解耦
独立部署
独立迭代
接口契约
服务治理

游戏服务端例子：

账号服务、角色服务、商城服务、活动服务、排行榜服务、邮件服务、支付服务

这些服务可以按业务边界独立开发、部署和扩容。

4.2 弹性原则

弹性是指系统部署规模可以随着业务量变化自动伸缩。

弹性包括：

- 横向扩容：增加服务实例。
- 横向缩容：流量下降时减少实例。

- 自动调度：把实例调度到合适节点。
- 按需使用资源：减少资源浪费。

考试关键词：

自动扩缩容
资源调度
负载均衡
按需分配
高峰支撑

游戏例子：

新活动开启时，登录、活动、排行榜服务压力升高；
云原生平台可以根据 CPU、QPS 或队列长度自动扩容相关服务。

4.3 可观测原则

可观测性是指系统运行状态可以被看见、分析和定位。

云原生中常用三件套：

Logging：日志
Tracing：链路追踪
Metrics：指标度量

三者区别：

项目	作用	例子
Logging	记录事件和错误详情	登录失败日志、支付回调日志
Tracing	追踪一次请求经过哪些服务	登录请求经过网关、账号服务、角色服务
Metrics	用数字衡量系统状态	QPS、RT、CPU、内存、错误率

考试关键词：

日志
链路追踪
指标度量
APM
调用链
故障定位
容量评估

可观测性常用于案例题回答：

通过日志、链路追踪和指标监控，对请求耗时、错误率、资源使用率进行监控，
及时发现性能瓶颈和故障点。

4.4 韧性原则

韧性是指系统面对异常时仍能维持服务能力，并能快速恢复。

异常可能包括：

- 节点宕机。
- 网络抖动。
- 服务超时。

- 下游服务异常。
- 流量突增。
- 数据库短暂不可用。

常见措施：

限流
熔断
降级
重试
超时控制
隔离
故障转移
自动恢复
多副本部署

考试关键词：

容错
高可用
故障隔离
自动恢复
抵御异常

注意：

韧性不是永远不出错，而是出错后影响范围可控、恢复速度快。

4.5 所有过程自动化原则

云原生强调从开发、测试、构建、部署、发布到运维都尽量自动化。

典型能力：

CI/CD
自动构建
自动测试
自动部署
自动扩缩容
自动回滚
自动巡检
基础设施即代码

考试关键词：

DevOps
CI/CD
声明式配置
自动化交付
蓝绿发布
灰度发布
金丝雀发布

为什么重要：

微服务数量多，部署节点多，如果没有自动化，运维复杂度会急剧上升。

4.6 零信任原则

零信任原则是指不默认信任任何网络内部或外部的用户、设备、系统和服务。

传统思路：

内网是可信的，外网是不可信的。

零信任思路：

任何访问都要认证、授权、校验和审计。

常见措施：

- 身份认证。
- 权限控制。
- 服务间认证。
- 加密通信。
- 最小权限。
- 安全审计。
- 访问策略控制。

考试关键词：

认证
授权
最小权限
服务间安全
访问控制
安全审计

4.7 架构持续演进原则

云原生架构不是一次设计完成后永远不变，而是要根据业务、技术和组织持续演进。

原因：

- 业务变化快。
- 系统规模增长。
- 技术栈更新。
- 团队结构变化。
- 存量系统需要渐进式改造。

考试关键词：

持续演进
渐进式迁移
架构治理
风险控制
技术债治理

5. 云原生主要架构模式

5.1 服务化架构模式

服务化架构模式要求按应用模块或业务领域划分服务，通过接口契约定义服务关系，通过 HTTP、REST、gRPC、消息队列等标准协议进行通信。

常搭配：

- DDD
- 微服务
- 接口契约
- 容器化部署
- 自动化测试
- CI/CD

考试要点：

服务化不是随便拆服务，而是按业务边界拆分。

5.2 Mesh 化架构模式

Mesh 化架构通常指服务网格思路。

核心思想：

把服务间通信、安全、流量控制、熔断、限流、可观测等通用能力，从业务代码或 SDK 中下沉到基础设施层。

传统方式：

业务代码 + RPC SDK + Redis SDK + MQ SDK + DB SDK

Mesh 方式：

业务代码尽量保持简单；
服务发现、负载均衡、限流、熔断、流量治理、安全策略交给 Sidecar / Proxy / 控制平面。

常见技术：

- Service Mesh
- Istio
- Linkerd
- Consul
- Sidecar
- Control Plane
- Data Plane

考试关键词：

- 服务间通信治理
- 基础设施下沉
- 业务无侵入
- 流量控制
- 安全策略
- 可观测

5.3 Serverless 模式

Serverless 不是没有服务器，而是开发者不用关心服务器。

核心思想：

开发者只关注函数或业务逻辑；
云平台负责运行环境、弹性伸缩、调度、可用性和按量计费。

适合场景：

- 事件驱动任务。
- 定时任务。
- 数据处理。
- 请求处理时间较短的接口。
- 峰值不稳定、低频触发的业务。

不适合场景：

- 长时间运行的复杂服务。
- 强状态、强连接、复杂会话系统。
- 对启动延迟极其敏感的核心链路。
- 服务之间复杂同步调用特别多的系统。

考试关键词：

事件触发
函数计算
自动伸缩
按量计费
无需管理服务器

5.4 存储计算分离模式

存储计算分离是指计算资源和存储资源独立扩展。

传统方式：

应用和状态紧密绑定，本地磁盘、本地会话、本地文件导致扩缩容困难。

云原生方式：

计算节点尽量无状态；
状态放到数据库、对象存储、缓存、消息队列、分布式存储等独立服务中。

优势：

- 计算服务可快速扩缩容。
- 故障实例可直接替换。
- 存储能力独立扩展。
- 更利于高可用和容灾。

考试关键词：

无状态服务
状态外置
弹性扩展

存储独立扩容
CAP

5.5 分布式事务模式

微服务拆分后，一个业务流程可能跨多个服务和数据库，会出现分布式事务问题。

常见模式：

模式	特点	优缺点
XA / 2PC	强一致，两阶段提交	一致性强，性能差，阻塞风险高
BASE / 最终一致性	基本可用、软状态、最终一致	性能好，允许短暂不一致
TCC	Try-Confirm-Cancel	隔离性可控，但业务侵入强
Saga	每步都有补偿事务	适合长事务，但补偿逻辑复杂
AT 模式	框架托管二阶段	开发成本低，但有场景限制

考试关键词：

强一致
最终一致
补偿事务
两阶段提交
性能与一致性权衡

5.6 可观测架构

可观测架构强调系统不仅要能运行，还要能被观测、分析和治理。

三件套：

Logging
Tracing
Metrics

案例题常用表达：

建立统一日志平台、分布式链路追踪和指标监控体系，
对接口耗时、错误率、吞吐量、资源使用率进行实时监控，
提升故障发现、定位和容量规划能力。

5.7 事件驱动架构 EDA

事件驱动架构通过事件进行服务间解耦。

适合场景：

- 数据变化通知。
- 异步解耦。
- 削峰填谷。
- 提升韧性。
- 事件流处理。
- CQRS。

例子：

玩家支付成功 -> 订单服务发布支付成功事件
背包服务订阅事件 -> 发放道具
邮件服务订阅事件 -> 发送通知
数据服务订阅事件 -> 记录运营数据

考试关键词：

异步
解耦
事件通知
消息队列
最终一致性
削峰填谷

6. 云原生相关技术

6.1 容器技术

容器是标准化的软件交付单元。

它把应用和依赖一起打包，使应用可以在不同环境中快速、可靠运行。

容器解决的问题：

- 开发环境、测试环境、生产环境不一致。
- 依赖版本冲突。
- 部署过程复杂。
- 应用迁移困难。

考试关键词：

Docker
镜像
容器
标准化交付
环境一致性
轻量级隔离
快速启动

容器不是虚拟机：

对比项	虚拟机	容器
隔离层次	硬件/操作系统级虚拟化	操作系统进程级隔离
启动速度	较慢	较快
资源开销	较大	较小
交付对象	VM 镜像	容器镜像
适合场景	强隔离、完整 OS	微服务、快速部署、弹性伸缩

6.2 容器编排

容器多了以后，需要平台统一管理，这就是容器编排。

容器编排能力：

- 资源调度。

- 应用部署与管理。
- 自动修复。
- 服务发现。
- 负载均衡。
- 弹性伸缩。
- 声明式 API。
- 可扩展架构。
- 可移植性。

典型技术：

Kubernetes，简称 K8s

考试关键词：

调度
编排
自愈
服务发现
负载均衡
声明式 API
弹性伸缩

6.3 微服务

微服务是云原生常见应用形态，但云原生不等于微服务。

微服务特点：

- 小服务。
- 单一职责。
- 独立部署。
- 独立扩展。
- 独立技术栈。
- 通过 API 或消息通信。
- 团队自治。

微服务设计约束：

个体约束：每个微服务按业务领域独立。
横向约束：服务之间要解决发现、调用、交互、治理。
纵向约束：服务私有数据只能通过该服务 API 访问。
全局约束：需要自动化 CI/CD、监控、发布策略、治理体系。

考试特别容易考：

微服务的数据不要被其他服务直接访问；
其他服务应通过当前微服务提供的 API 访问其数据。

6.4 服务网格 Service Mesh

服务网格把服务治理能力从业务代码中抽离出来。

主要能力：

- 服务发现。
- 负载均衡。
- 熔断限流。
- 流量路由。
- 灰度发布。
- 服务间认证。
- 加密通信。
- 可观测性。

典型结构：

控制平面：负责配置、策略、治理规则。
数据平面：通常由 Sidecar 代理组成，处理真实服务流量。

考试关键词：

Sidecar
控制平面
数据平面
流量治理
服务间通信
业务无侵入

6.5 DevOps / DevSecOps

DevOps 强调开发和运维协同，提升交付效率。

DevSecOps 在 DevOps 中融入安全，强调安全左移和全流程安全。

常见能力：

代码提交
自动构建
自动测试
镜像构建
安全扫描
自动部署
灰度发布
监控反馈
自动回滚

考试关键词：

持续集成
持续交付
持续部署
安全左移
自动化流水线

7. 云原生架构的优点

7.1 敏捷

服务拆分后，不同团队可以并行开发、测试和发布，业务响应更快。

7.2 弹性

系统可以根据业务压力自动扩缩容，适合峰值明显的互联网业务。

7.3 高可用

通过多副本、健康检查、自动拉起、故障转移等方式提升可用性。

7.4 可观测

通过日志、链路追踪和指标，快速定位问题。

7.5 自动化

通过 CI/CD、容器编排、声明式配置减少人工操作风险。

7.6 成本优化

按需使用资源，缩容闲置实例，减少资源浪费。

8. 云原生架构的风险和反模式

考试很喜欢考“不是所有系统都适合直接拆成微服务”。

8.1 庞大的单体应用

问题：

- 代码耦合。
- 模块边界不清。
- 变更影响范围大。
- 无法按模块独立扩容。
- 一个模块故障可能影响整体。

改进：

先梳理业务边界，再渐进式服务化。

8.2 单体应用硬拆为微服务

这是典型错误。

表现：

- 模块本来高度耦合，却强行拆分。
- 服务拆了，但数据库没拆。
- 服务之间大量同步调用。

- 性能下降。
- 分布式事务复杂度上升。
- 运维成本增加。

记忆：

微服务不是越多越好，边界不清就不要硬拆。

8.3 缺乏自动化能力的微服务

微服务数量多，如果没有自动化部署、监控、治理，会导致：

- 发布复杂。
- 排障困难。
- 运维成本上升。
- 版本不一致。
- 故障范围难以判断。

考试表达：

微服务必须配套自动化交付、监控、服务治理和运维体系，否则复杂度会超过收益。

9. 常见考点

9.1 选择题考点

高频点：

云原生定义
云原生不是简单上云
云原生七大原则
容器与虚拟机区别
容器编排能力
微服务设计约束
服务网格作用
Serverless 特点和适用场景
可观测三件套
分布式事务模式
云原生反模式

常见判断：

说法	对错	原因
把系统部署到云服务器就是云原生	错	只是上云，不一定云原生
用 Docker 就一定是云原生	错	容器只是云原生技术之一
云原生可以把所有非功能需求完全交给云平台	错	是最大化剥离，不是完全不用设计
微服务越细越好	错	粒度过细会增加调用、事务和运维复杂度
服务私有数据可被其他服务直接访问	错	应通过该服务 API 访问
Logging、Tracing、Metrics 是可观测的重要组成	对	对应日志、链路、指标
Serverless 是没有服务器	错	是开发者无须管理服务器
服务网格用于服务间通信、安全、流量控制和可观测	对	通用治理能力下沉到基础设施

9.2 案例题考点

案例题通常不会问“云原生是什么”，而是给一个系统背景，让你设计或改造。

常见问法：

如何提升系统弹性？
如何实现灰度发布？
如何提高可观测性？
如何进行微服务拆分？
如何解决服务间通信和治理？
如何处理分布式事务？
如何提高系统韧性？
如何从单体迁移到云原生？

答题模板：

1. 服务化拆分：按业务领域划分服务，明确服务边界和接口契约。
2. 容器化部署：将应用和依赖打包为镜像，实现环境一致性和快速部署。
3. 容器编排：使用编排平台进行资源调度、服务发现、负载均衡、自动恢复和弹性伸缩。
4. 自动化交付：建立 CI/CD 流水线，支持自动构建、测试、部署、灰度和回滚。
5. 可观测体系：建设日志、链路追踪和指标监控，支撑故障定位和容量评估。
6. 韧性设计：采用限流、熔断、降级、重试、超时、隔离等机制控制故障影响范围。
7. 安全治理：采用零信任思想，实施认证、授权、加密通信、最小权限和审计。
8. 数据与事务：按服务边界隔离数据，对跨服务事务采用最终一致性、TCC、Saga 等方案。

9.3 论文考点

云原生可以套到这些论文题：

论云原生架构及其应用
论微服务架构及其应用
论系统高可用架构设计
论系统性能优化设计
论系统可观测性设计
论自动化运维及其应用
论 Serverless 架构模式
论服务网格技术及其应用
论分布式事务及其解决方案
论基于云原生数据库的系统架构设计

论文中不要只写技术名词，要写：

项目背景 -> 架构问题 -> 设计方案 -> 实施措施 -> 效果 -> 反思

10. 用你的游戏项目怎么理解

你是 Unity / Cocos 手机游戏开发工程师，熟悉 Python、Orleans 服务端和 MySQL，可以这样映射云原生。

10.1 传统游戏服务端问题

登录高峰时服务压力大
活动开启时请求突增
排行榜和商城是热点模块
支付和道具发放需要一致性
多款游戏版本迭代快
弱网重连导致请求波动
服务端故障影响玩家体验

10.2 云原生改造思路

账号、角色、商城、活动、排行榜、邮件、支付拆成服务

服务容器化部署
用编排平台自动拉起和扩缩容
活动高峰前预扩容，活动中自动扩容
支付成功通过事件驱动通知背包、邮件和数据统计
对排行榜查询加缓存和限流
通过日志、链路追踪、指标监控定位慢请求
用灰度发布控制新版本风险
跨服务发奖采用消息最终一致性或 TCC

10.3 可写进论文的项目表达

在项目中，我们将账号、角色、商城、活动、排行榜、邮件等模块进行服务化拆分，并通过容器化方式统一交付。平台通过容器编排能力实现服务实例的自动调度、健康检查、故障自愈和弹性伸缩。在运营活动高峰期，根据 QPS、CPU 利用率和消息队列积压情况动态扩容活动服务、登录服务和排行榜服务。同时建设统一日志、链路追踪和指标监控体系，用于定位跨服务调用中的性能瓶颈。对于支付发奖等跨服务一致性场景，采用消息驱动的最终一致性和幂等处理机制，降低分布式事务对系统性能和可用性的影响。

11. 快速记忆

11.1 云原生一句话

云原生 = 应用为云而设计，把弹性、韧性、观测、安全、发布、治理交给平台化能力。

11.2 七大原则

服务化
弹性
可观测
韧性
全自动化
零信任
持续演进

记忆口诀：

服弹观韧，自零演

11.3 三件套

Logging：日志
Tracing：链路
Metrics：指标

11.4 三个反模式

大单体
硬拆微服务
没自动化的微服务

11.5 容器编排能力

调度、部署、自愈、发现、负载、伸缩、声明式、可移植

11.6 服务网格

服务网格 = 把服务间通信治理能力下沉到基础设施。

11.7 Serverless

Serverless = 不管服务器，事件触发，自动伸缩，按量计费。

12. 考前最低掌握清单

如果时间很紧，至少背下面这些：

1. 云原生不是简单上云，而是面向云环境的架构原则和技术体系。
2. 云原生目标是最大化剥离非业务代码，让云设施接管弹性、韧性、安全、可观测、灰度等能力。
3. 七大原则：服务化、弹性、可观测、韧性、所有过程自动化、零信任、架构持续演进。
4. 主要模式：服务化、Mesh、Serverless、存储计算分离、分布式事务、可观测、事件驱动。
5. 相关技术：容器、容器编排、微服务、Serverless、Service Mesh、DevOps。
6. 可观测三件套：Logging、Tracing、Metrics。
7. 微服务数据访问原则：服务私有数据必须通过该服务 API 访问。
8. 典型反模式：庞大单体、单体硬拆微服务、缺乏自动化能力的微服务。

13. 资料依据

- 本地教材：最新教材-新大纲/03、【带搜索】系统架构设计师第二版.pdf，第 14 章“云原生架构设计理论与实践”。
- 本地冲刺资料：最新教材-新大纲/04、考试32小时通关-第2版（新版）.pdf，第 17 小时“云原生架构设计理论与实践”。
- CNCF 对 Cloud Native 的公开定义：<https://github.com/cncf/toc/blob/main/DEFINITION.md>。